Sampling and response noise epistemic uncertainties: from linear regressors to linearized deep networks

Pierre Nodet Thomas George

Orange Research

EurIPS
2025 Workshop on
Epistemic Intelligence in Machine Learning

TL;DR

Where does epistemic uncertainty for a test prediction stem from ?

- aleatoric noise $\varepsilon|X$?
- joint sampling of (X, ε) ?

$\Rightarrow$ Uncertainty due to finite sampling is shaped by the choice of hypothesis class (here, linear models and deep networks).
This work: Insights from stats 101 concepts and linearization of deep network training dynamics.

Linear Regression

Linear data generating process:

$$y = w^{*\top} x + \varepsilon \qquad \text{with } x \sim \mathcal{N}(0, \Sigma),\ \varepsilon \sim \mathcal{N}(0, \sigma^2) \text{ and } w^* \in \mathbb{R}^d$$

- Admits a closed-form solution with Ordinary Least Squares (OLS)
- Prediction on test point x_{test}:

$$\hat{y}(x_{\text{test}}, X, \varepsilon) = x_{\text{test}}^\top w^* + \underbrace{x_{\text{test}}^\top (X^\top X)^{-1} X^\top \varepsilon}_{\text{depends on } X, \varepsilon}$$

Uncertainty and variance

Had we obtained a different finite sample (X, ε), we would have obtained a different test prediction $\Rightarrow$ epistemic uncertainty

- Variance due to response noise $\varepsilon|X$

$$\text{var}\left(\hat{y}(x_{\text{test}}, X, \varepsilon)\,|X\right) = \text{var}\left(x_{\text{test}}^\top (X^\top X)^{-1} X^\top \varepsilon | X\right)$$

- Variance due to sampling (X, ε) (or total variance)

$$\text{var}\left(\hat{y}(x_{\text{test}}, X, \varepsilon)\right) = \text{var}\left(x_{\text{test}}^\top (X^\top X)^{-1} X^\top \varepsilon\right)$$

Variance estimators from a single sample (X, ε)

- Closed-form expression of the variance (parametric)

$$\hat{\text{var}}_{\text{OLS}}\left(\hat{y}(x_{\text{test}}, X, \varepsilon)\,|X\right) = \hat{\sigma}^2_{\text{OLS}} x_{\text{test}}^\top (X^\top X)^{-1} x_{\text{test}}$$

using the OLS estimator $\hat{\sigma}^2_{\text{OLS}} = \frac{1}{n-d} \sum_{i=1}^n \left(y_i - \hat{y}(x_i)\right)^2$

- Jackknife estimator of the total variance (non-parametric)

$$\hat{\text{var}}_{\text{JK}}(\hat{y}(x_{\text{test}}, X, \varepsilon)) = \sum_{i=1}^n \left(\hat{y}(x_{\text{test}}) - \hat{y}^{(-i)}(x_{\text{test}})\right)^2$$

where $\hat{y}^{(-i)}(x_{\text{test}})$ is the prediction on x_{test} of the regressor trained without the training example x_i

- Closed-form expression of the Jackknife estimator from Sherman-Morrison

$$\hat{y}(x_{\text{test}}) - \hat{y}^{(-i)}(x_{\text{test}}) = \frac{1}{1 - h_{ii}} x_{\text{test}}^\top (X^\top X)^{-1} x_i \hat{e}_i$$

where $h_{ii} = x_i^\top (X^\top X)^{-1} x_i$ is the leverage of x_i and $\hat{e}_i = y_i - \hat{y}(x_i)$ is the residual of the model $\hat{y}$ at x_i.

$$\hat{\text{var}}_{\text{JK}}\left(\hat{y}(x_{\text{test}}, X, \varepsilon)\right) = x_{\text{test}}^\top (X^\top X)^{-1} \sum_{i=1}^n \left(\frac{\hat{e}_i^2}{(1 - h_{ii})^2} x_i^\top x_i\right) (X^\top X)^{-1} x_{\text{test}}$$

Deep networks linearization

Nonlinear data generating process;

$$y = f_{w^*}(x) + \varepsilon$$

with $x \sim \mathcal{N}(0, \Sigma)$, $\varepsilon \sim \mathcal{N}(0, \sigma^2)$ and $w^* \in \mathbb{R}^d$.

- f_w is a neural network with parameters w (all weights and biases)
- No closed-form solution solved with iterative solvers (SGD)
- Linearize f_w at learned parameters $\hat{w}$, used as anchor

$$f_w(\cdot) = f_{\hat{w}}(\cdot) + \delta w^\top \phi_{\hat{w}}(\cdot) + \text{H.O.T.}$$

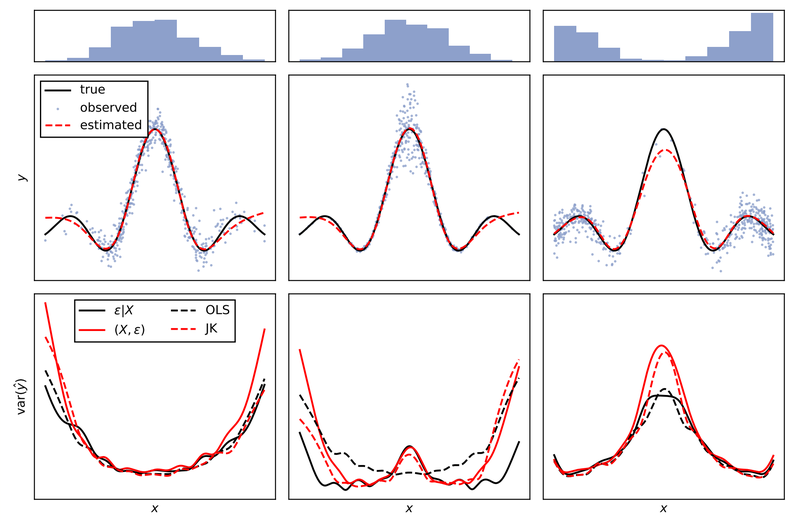
where $\phi_{\hat{w}}(\cdot) = \left.\frac{\partial f_w(\cdot)}{\partial w}\right|_{w=\hat{w}}$ are called the tangent features (Jacobian)

- Use linear regression estimators by analogy:

$$X \to \Phi_{\hat{w}} \quad x_{\text{test}} \to \phi_{\hat{w}}(x_{\text{test}})$$

Synthetic data regression

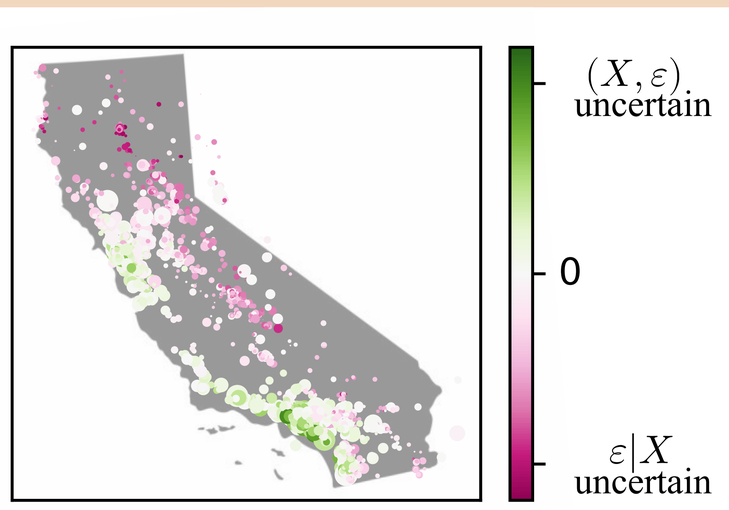
Setting: $x \sim \mathcal{N}(0, I) \in \mathbb{R}^6$ (response $y = f(x_1)$ only, other directions are used to increase task difficulty). one-hidden layer MLP, $\tanh$ activation, trained with full batch L-BFGS, and ℓ_2 regularization.



Variance: Monte-Carlo estimate (100 runs) in solid line, our estimators in dashed line. Plots show 5 runs averaged.
(left) normally distributed samples, homoscedastic noise
(center) normally distributed samples, heteroscedastic noise
(right) non-normally distributed samples, homoscedastic noise

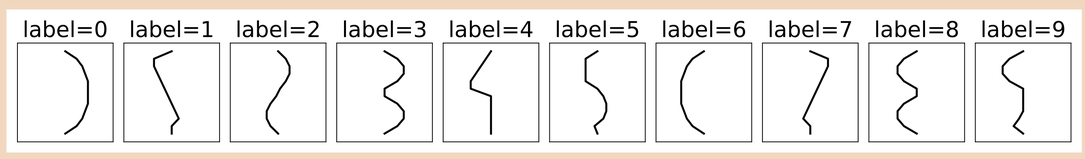
California housing regression

Setting: one-hidden layer MLP, $\tanh$ activation, trained with full batch L-BFGS, and ℓ_2 regularization.



Absolute error (circle size) and variance estimators (circle color) on test examples.
Green districts: high value disparities, requires increasing sample size.
Pink districts: few districts/poorer districts, more accurate labeling is recommended.

Proof of concept: MNIST-1D classification



Setting: 1D ReLU CNN, trained with Adam and weight decay.

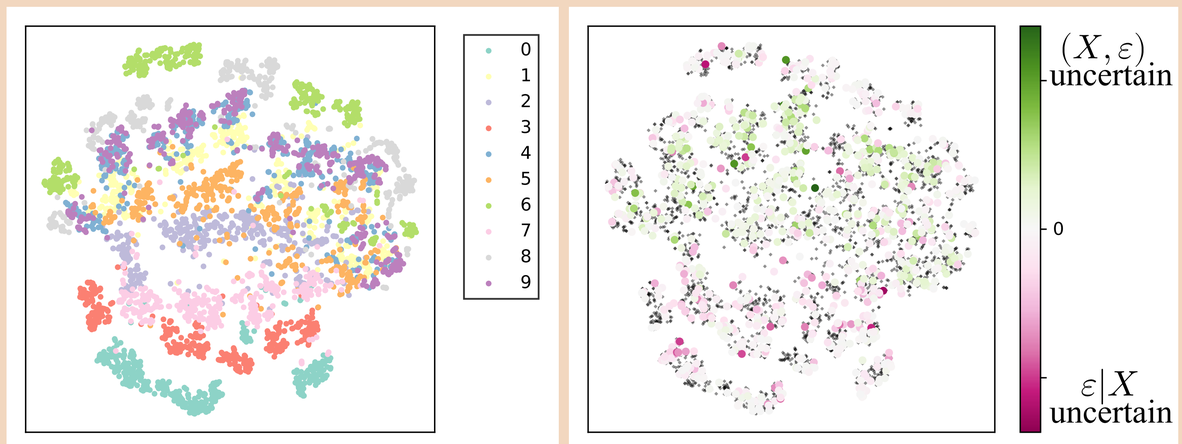


Figure: t-SNE on the embeddings of the last layer. **(left)** training examples classes **(right)** training examples (black dots) and variance estimators **(in green)** regions where classes overlap, increasing the sample size would help discriminate classes **(in pink)** examples closer to class boundaries, more affected by training labeling noise.

Future work: Scaling to deep networks

$$\phi_{\mathbf{w}}(x_{\text{test}})^\top \underbrace{\left(\Phi_{\mathbf{w}}^\top \Phi_{\mathbf{w}}\right)^{-1}}_{\text{FIM}} \phi_{\mathbf{w}}(x_{\text{test}})$$

- FIM is $d \times d$: $\mathcal{O}(d^2)$ memory, $\mathcal{O}(d^3)$ inverse
  — inverse Hessian vector product using EKFAC
- Quality of linearized estimator using $\phi_{\hat{w}}(\cdot)$
- Classification (IRLS, delta method, ...)

Paper